

Disciplina: Automação Inteligente

ATIVIDADE PRÁTICA 3A – LÓGICA FUZZY

Controle Fuzzy para Desvio de Obstáculos em Robótica Móvel

Projeto, cálculo manual e implementação computacional de um sistema
Mamdani

Aluno: Fernando Roque Caldas de Oliveira
Matrícula: 202211130038
Software: Scilab 6.1.1 (Windows 64 bits)
Módulo: sciFLT 0.5

Sumário

1	Objetivo	1
2	Variáveis linguísticas	1
2.1	Distância frontal	1
2.2	Assimetria lateral	1
2.3	Ângulo de direção	2
3	Base de regras	3
4	Cálculo manual para $d = 10$ cm e $a = -10$ cm	3
4.1	Fase 1 – Fuzzificação	3
4.2	Fase 2 – Regras ativadas	4
4.3	Fase 3 – Conectivo E e implicação	4
4.4	Fase 4 – Agregação	4
4.5	Fase 5 – Defuzzificação	4
5	Implementação computacional	5
6	Superfície tridimensional de controle	6
7	Execução no Windows	7
8	Verificação dos itens solicitados	8
9	Conclusão	8
A	Código-fonte completo em Scilab	9

1 Objetivo

O objetivo desta atividade é projetar e implementar um controlador fuzzy reativo para um robô móvel. O sistema recebe a distância frontal até um obstáculo e a assimetria entre os sensores laterais. A saída é um ângulo de correção que permite desviar do obstáculo de maneira suave.

Foi utilizado um sistema Mamdani com conectivo E pelo mínimo, implicação pelo mínimo, agregação pelo máximo e defuzzificação pelo centroide. A implementação foi realizada exclusivamente no Scilab 6.1.1 para Windows, com o módulo sciFLT 0.5.

2 Variáveis linguísticas

2.1 Distância frontal

A distância frontal d pertence ao universo $[0, 100]$ cm e possui os termos Perto, Média e Longe.

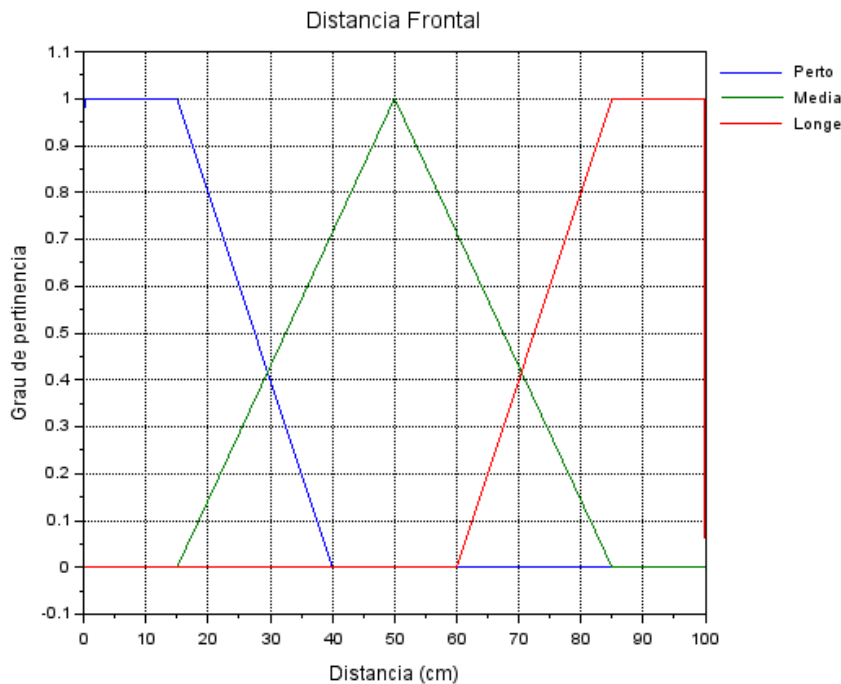


Figura 1: Funções de pertinência da distância frontal.

2.2 Assimetria lateral

A assimetria é definida por $a = d_{dir} - d_{esq}$ e pertence ao universo $[-50, 50]$ cm. Valores negativos indicam obstáculo mais próximo do lado direito. Os termos são Negativa, Zero e Positiva.

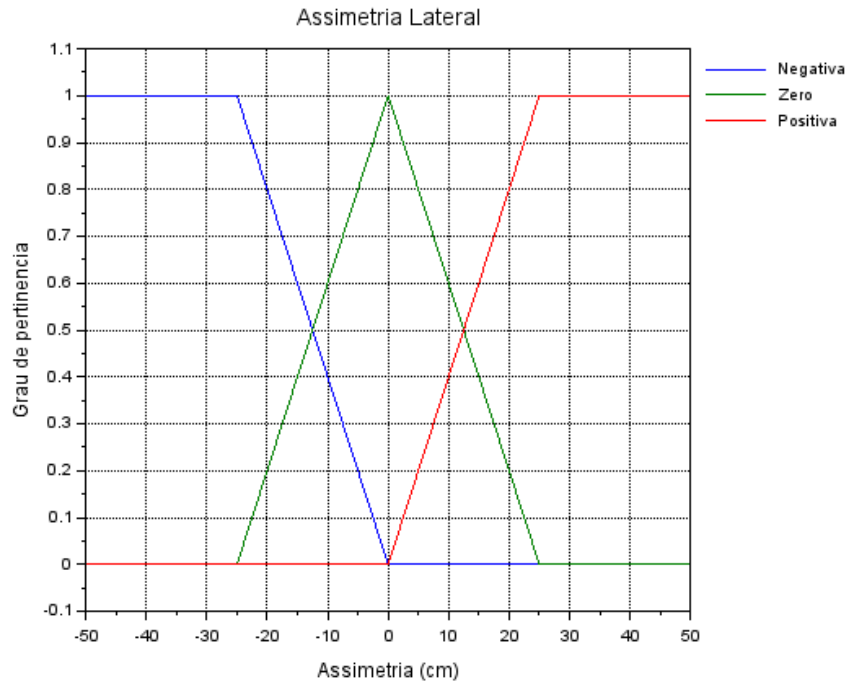


Figura 2: Funções de pertinência da assimetria lateral.

2.3 Ângulo de direção

O ângulo θ pertence ao universo $[-45^\circ, 45^\circ]$. Valores negativos indicam giro para a esquerda e valores positivos indicam giro para a direita. Os termos são Virar à Esquerda, Seguir em Frente e Virar à Direita.

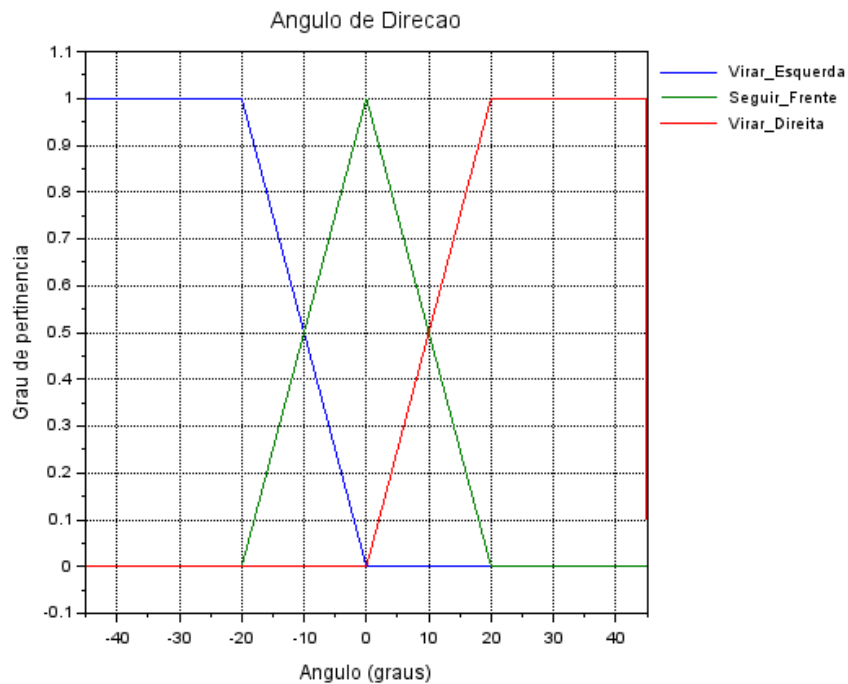


Figura 3: Funções de pertinência do ângulo de direção.

3 Base de regras

Todas as regras utilizam o conectivo E. A base completa possui nove regras.

Tabela 1: Base de regras do controlador fuzzy.

Distância / Assimetria	Negativa	Zero	Positiva
Perto	Virar à esquerda	Virar à direita	Virar à direita
Média	Virar à esquerda	Seguir em frente	Virar à direita
Longe	Seguir em frente	Seguir em frente	Seguir em frente

4 Cálculo manual para $d = 10$ cm e $a = -10$ cm

4.1 Fase 1 – Fuzzificação

Para a distância frontal, $d = 10$ cm está no trecho constante da função Perto ($0 \leq d \leq 15$), antes do suporte da função Média ($d < 15$) e antes do suporte da função Longe ($d < 60$). Portanto:

$$\mu_{\text{Perto}}(10) = 1, \quad \mu_{\text{Média}}(10) = 0, \quad \mu_{\text{Longe}}(10) = 0.$$

Para a assimetria lateral, $a = -10$ cm pertence simultaneamente aos trechos lineares de Negativa e Zero. Usando as equações das retas indicadas no gráfico:

$$\mu_{\text{Negativa}}(-10) = \frac{0 - a}{0 - (-25)} = \frac{0 - (-10)}{25} = \frac{10}{25} = 0,4,$$

$$\mu_{\text{Zero}}(-10) = \frac{a - (-25)}{0 - (-25)} = \frac{-10 + 25}{25} = \frac{15}{25} = 0,6,$$

e $\mu_{\text{Positiva}}(-10) = 0$, pois a função Positiva só possui suporte a partir de $a = 0$.

Os seis campos solicitados na atividade ficam, portanto, preenchidos por:

$$\mu_{\text{Perto}}(10) = 1,0 \quad \mu_{\text{Média}}(10) = 0,0 \quad \mu_{\text{Longe}}(10) = 0,0$$

$$\mu_{\text{Negativa}}(-10) = 0,4 \quad \mu_{\text{Zero}}(-10) = 0,6 \quad \mu_{\text{Positiva}}(-10) = 0,0.$$

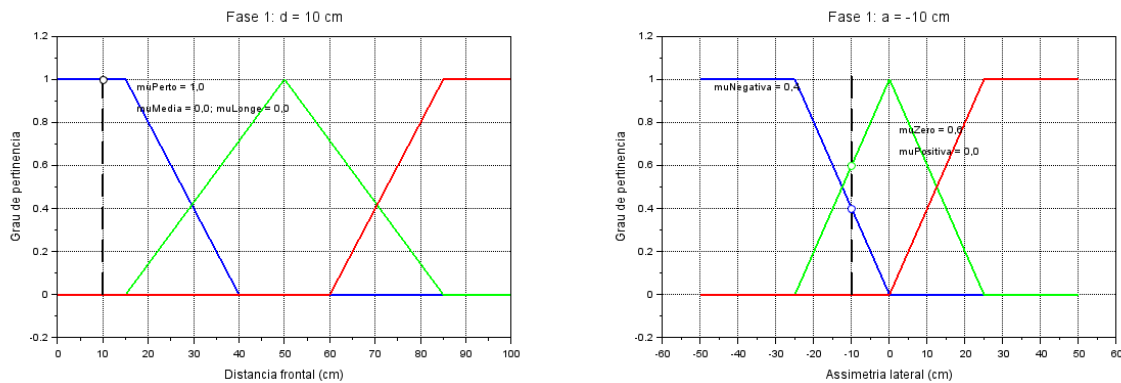


Figura 4: Fase 1 preenchida: valores crisp marcados e graus de pertinência.

4.2 Fase 2 – Regras ativadas

Somente duas regras apresentam antecedente diferente de zero:

1. Se a distância é Perto e a assimetria é Negativa, então Virar à Esquerda.
2. Se a distância é Perto e a assimetria é Zero, então Virar à Direita.

4.3 Fase 3 – Conectivo E e implicação

Pelo operador mínimo:

$$\alpha_1 = \min(1; 0,4) = 0,4, \quad \alpha_2 = \min(1; 0,6) = 0,6.$$

A implicação de Mamdani trunca Virar à Esquerda em 0,4 e Virar à Direita em 0,6.

4.4 Fase 4 – Agregação

As saídas truncadas são combinadas pelo máximo:

$$\mu_{agregada}(\theta) = \max(\min(0,4; \mu_{Esquerda}(\theta)), \min(0,6; \mu_{Direita}(\theta))).$$

A Figura 5 reúne o que deve ser desenhado no gráfico em branco das Fases 3 e 4: a contribuição da Regra 1 truncada em 0,4, a contribuição da Regra 2 truncada em 0,6 e o contorno agregado obtido pelo máximo.



Figura 5: Fases 3 e 4 preenchidas: implicação Mamdani e agregação.

4.5 Fase 5 – Defuzzificação

O universo de saída foi discretizado em passos de 5° , exatamente como solicitado. Em cada ponto calcula-se primeiro $\mu_{agregada}(\theta)$ e depois o produto $\mu_{agregada}(\theta)\theta$.

Tabela 2: Valores usados no cálculo do Centro de Área.

θ	μ	$\mu\theta$	θ	μ	$\mu\theta$
-45	0,40	-18,00	5	0,25	1,25
-40	0,40	-16,00	10	0,50	5,00
-35	0,40	-14,00	15	0,60	9,00
-30	0,40	-12,00	20	0,60	12,00
-25	0,40	-10,00	25	0,60	15,00
-20	0,40	-8,00	30	0,60	18,00
-15	0,40	-6,00	35	0,60	21,00
-10	0,40	-4,00	40	0,60	24,00
-5	0,25	-1,25	45	0,60	27,00
0	0,00	0,00			

Memorial de cálculo manual do CDA

No lado esquerdo, existem oito amostras com pertinência 0,40 entre -45° e -10° , além da amostra -5° com pertinência 0,25. No lado direito, as pertinências são 0,25 em 5° , 0,50 em 10° e 0,60 nas sete amostras de 15° a 45° . Assim, o denominador do CDA é:

$$\begin{aligned}\sum \mu(\theta) &= \underbrace{(8 \cdot 0,40 + 0,25)}_{3,45} + \underbrace{(0,25 + 0,50 + 7 \cdot 0,60)}_{4,95} \\ &= 3,45 + 4,95 = 8,40.\end{aligned}$$

O numerador, incluindo os sinais dos ângulos, é:

$$\begin{aligned}\sum \mu(\theta)\theta &= \underbrace{0,40(-45 - 40 - 35 - 30 - 25 - 20 - 15 - 10)}_{-89,25} + 0,25(-5) \\ &\quad + \underbrace{0,25(5) + 0,50(10) + 0,60(15 + 20 + 25 + 30 + 35 + 40 + 45)}_{132,25} \\ &= -89,25 + 132,25 = 43,00.\end{aligned}$$

Substituindo os dois somatórios na fórmula do Centro de Área:

$$\theta_{manual} = \frac{\sum \mu(\theta)\theta}{\sum \mu(\theta)} = \frac{43,00}{8,40} = 5,1190^\circ$$

O resultado é positivo, portanto a decisão é **virar para a direita**.

5 Implementação computacional

O sistema foi implementado com as funções `newfls`, `addvar`, `addmf`, `addrule`, `evalfls` e `savefls` do `sciFLT`. A avaliação computacional utiliza 1001 pontos no universo da saída.

Para $d = 10$ cm e $a = -10$ cm, o software produziu:

- ângulo: $\theta = 4,828622^\circ$;
- ação: virar para a direita;
- diferença em relação ao cálculo manual: $-0,290426^\circ$.

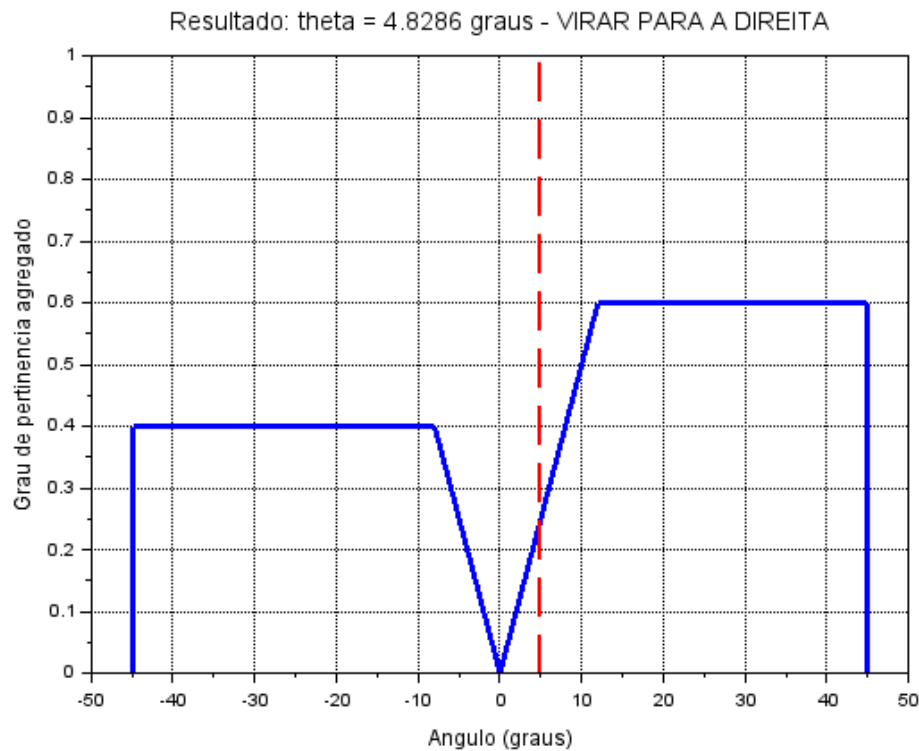


Figura 6: Saída agregada e ângulo defuzzificado pelo sciFLT.

A pequena diferença é esperada porque o cálculo manual utiliza passos de 5° , enquanto o programa emprega uma discretização muito mais fina.

6 Superfície tridimensional de controle

A Figura 7 apresenta o mapeamento completo entre as duas entradas e o ângulo de saída. A região azul representa correções para a esquerda, a região vermelha representa correções para a direita e a região próxima de zero corresponde ao deslocamento em frente.

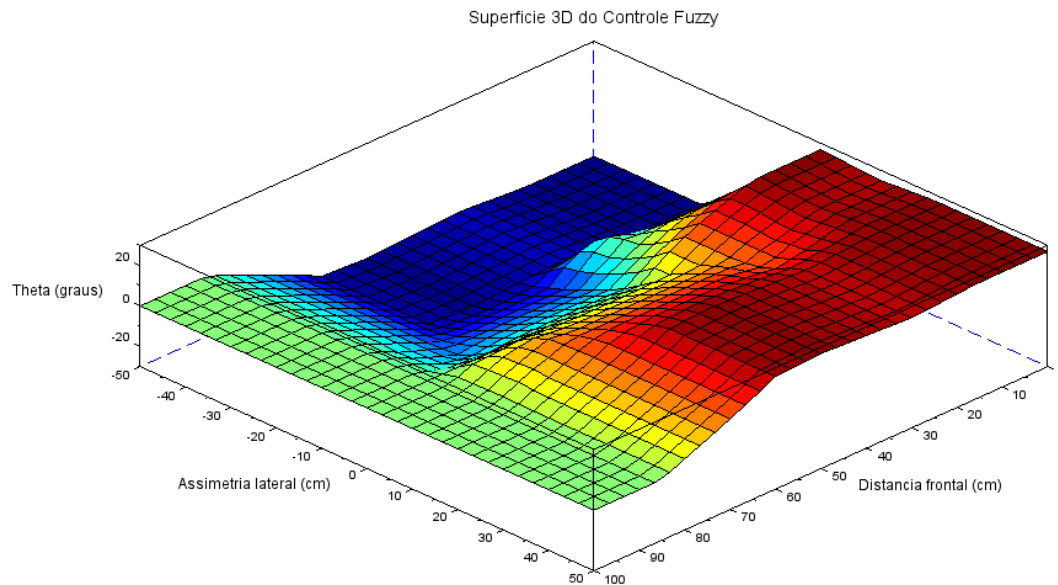


Figura 7: Superfície tridimensional do controlador fuzzy.

7 Execução no Windows

O projeto foi preparado para Windows 10 ou 11, em 64 bits, utilizando somente o Scilab 6.1.1 e o sciFLT 0.5. No console do Scilab, devem ser executados os comandos:

```
1 cd("C:/Users/ferna/Desktop/01 - Faculdade/Faculdade/10 semestre/AUTOMAÇÃO")
2 exec("fuzzy_robotica.sce", -1)
```

O código-fonte está em:

```
C:/Users/ferna/Desktop/01 - Faculdade/Faculdade/10 semestre/AUTOMAÇÃO/  
fuzzy_robotica.sce
```

O relatório, o arquivo do sistema fuzzy, o resultado numérico e os gráficos ficam na mesma pasta do projeto. As imagens são armazenadas na subpasta **prints**.

8 Verificação dos itens solicitados

Tabela 3: Conferência da entrega em relação ao enunciado da atividade.

Exigência	Atendimento
Cinco fases do cálculo manual	Seções 4.1 a 4.5, com equações, regras, implicação, agregação e CDA.
CDA em passos de 5°	Tabela completa de -45° a 45° e memorial dos dois somatórios.
Implementação computacional	Código Scilab/sciFLT, sistema salvo em <code>.fls</code> e nove regras.
Teste $d = 10$ e $a = -10$	Print da saída e valor computacional $4,828622^\circ$.
Comparação manual/computacional	$5,119048^\circ$ contra $4,828622^\circ$; diferença de $-0,290426^\circ$.
Documentação e GitHub	README completo, código, relatório e pasta <code>prints</code> organizados.

9 Conclusão

O controlador fuzzy foi implementado e validado com as nove regras especificadas. Para as entradas solicitadas, tanto o cálculo manual quanto a simulação computacional indicam giro para a direita. A diferença numérica é pequena e decorre exclusivamente da resolução usada na defuzzificação. A superfície tridimensional confirma transições suaves entre as ações de virar à esquerda, seguir em frente e virar à direita.

A Código-fonte completo em Scilab

```

1 // =====
2 // ATIVIDADE 3A - LOGICA FUZZY
3 // Controle Fuzzy para Desvio de Obstaculos em Robotica Movei
4 // Implementacao para Scilab 6.1.1 + sciFLT 0.5
5 // Autor: Fernando Roque Caldas de Oliveira
6 // Matricula: 202211130038
7 // =====
8
9 mode(-1);
10 clc;
11 clear;
12
13 // Carrega o Fuzzy Logic Toolbox do Scilab.
14 atomsLoad("sciFLT");
15
16 // Pasta em que este arquivo esta salvo.
17 pasta_projeto = get_absolute_file_path("fuzzy_robotica.sce");
18 pasta_prints = pasta_projeto + "prints/";
19
20 if ~isdir(pasta_prints) then
21     mkdir(pasta_prints);
22 end
23
24 // =====
25 // 1. CRIACAO DO SISTEMA FUZZY MAMDANI
26 // S-Norm = maximo; T-Norm = minimo; complemento = classico;
27 // defuzzificacao = centroide.
28 // =====
29
30 fls = newfls("m", "Controle_Robotica", "max", "min", "one", "centroide");
31 fls.comment = "Controle fuzzy Mamdani para desvio de obstaculos";
32 fls.ImpMethod = "min";
33 fls.AggMethod = "max";
34
35 // =====
36 // 2. ENTRADA 1 - DISTANCIA FRONTAL
37 // Universo: [0 100] cm
38 // =====
39
40 fls = addvar(fls, "input", "Distancia", [0 100]);
41 fls = addmf(fls, "input", 1, "Perto", "trapmf", [0 0 15 40]);
42 fls = addmf(fls, "input", 1, "Media", "trimf", [15 50 85]);
43 fls = addmf(fls, "input", 1, "Longe", "trapmf", [60 85 100 100]);
44
45 // =====
46 // 3. ENTRADA 2 - ASSIMETRIA LATERAL
47 // a = d_dir - d_esq; universo: [-50 50] cm
48 // =====
49
50 fls = addvar(fls, "input", "Assimetria", [-50 50]);
51 fls = addmf(fls, "input", 2, "Negativa", "trapmf", [-50 -50 -25 0]);
52 fls = addmf(fls, "input", 2, "Zero", "trimf", [-25 0 25]);
53 fls = addmf(fls, "input", 2, "Positiva", "trapmf", [0 25 50 50]);
54
55 // =====
56 // 4. SAIDA - ANGULO DE DIRECAO
57 // Universo: [-45 45] graus
58 // =====
59
60 fls = addvar(fls, "output", "Angulo", [-45 45]);
61 fls = addmf(fls, "output", 1, "Virar_Esquerda", "trapmf", [-45 -45 -20 0]);
62 fls = addmf(fls, "output", 1, "Seguir_Frente", "trimf", [-20 0 20]);
63 fls = addmf(fls, "output", 1, "Virar_Direita", "trapmf", [0 20 45 45]);
64
65 // =====
66 // 5. BASE DE REGRAS
67 // Formato por linha:
68 // [MF_distancia MF_assimetria MF_angulo conectivo peso]
69 // conectivo 1 = E (T-Norm); peso = 1.
70 // =====
71
72 regras = [
73     1 1 1 1 1; // Perto & Negativa -> Virar Esquerda
74     1 2 3 1 1; // Perto & Zero -> Virar Direita
75     1 3 3 1 1; // Perto & Positiva -> Virar Direita
76     2 1 1 1 1; // Media & Negativa -> Virar Esquerda
77     2 2 2 1 1; // Media & Zero -> Seguir em Frente
78     2 3 3 1 1; // Media & Positiva -> Virar Direita
79     3 1 2 1 1; // Longe & Negativa -> Seguir em Frente
80     3 2 2 1 1; // Longe & Zero -> Seguir em Frente
81     3 3 2 1 1 // Longe & Positiva -> Seguir em Frente
82 ];
83
84 fls = addrule(fls, regras);
85

```

```

86 // =====
87 // 6. TESTE SOLICITADO: d = 10 cm; a = -10 cm
88 // =====
89
90 d = 10;
91 a = -10;
92 n_pontos = 1001;
93
94 [theta, entradas_regras, saida_agregada] = evalfls([d a], fls, n_pontos);
95 theta = theta(1);
96
97 if theta < 0 then
98     acao = "VIRAR PARA A ESQUERDA";
99 elseif theta > 0 then
100     acao = "VIRAR PARA A DIREITA";
101 else
102     acao = "SEGUIR EM FRENTE";
103 end
104
105 mprintf("\n===== \n");
106 mprintf("                RESULTADO DO SISTEMA FUZZY \n");
107 mprintf("===== \n");
108 mprintf("Distancia frontal = %.2f cm \n", d);
109 mprintf("Assimetria lateral = %.2f cm \n", a);
110 mprintf("Angulo calculado = %.6f graus \n", theta);
111 mprintf("Acao: %s \n", acao);
112 mprintf("===== \n \n");
113
114 // =====
115 // 7. CALCULO MANUAL DISCRETIZADO EM PASSOS DE 5 GRAUS
116 // =====
117
118 mu_perto = trapmf(d, [0 0 15 40]);
119 mu_media = trimf(d, [15 50 85]);
120 mu_longe = trapmf(d, [60 85 100 100]);
121
122 mu_negativa = trapmf(a, [-50 -50 -25 0]);
123 mu_zero = trimf(a, [-25 0 25]);
124 mu_positiva = trapmf(a, [0 25 50 50]);
125
126 forca_regra_1 = min(mu_perto, mu_negativa);
127 forca_regra_2 = min(mu_perto, mu_zero);
128
129 theta_discreto = -45:5:45;
130 mf_esquerda = trapmf(theta_discreto, [-45 -45 -20 0]);
131 mf_direita = trapmf(theta_discreto, [0 20 45 45]);
132
133 // O trapmf do sciFLT retorna zero exatamente nos extremos quando a=b ou c=d.
134 // Nos ombros definidos pela atividade, esses extremos pertencem ao patamar 1.
135 mf_esquerda(1) = 1;
136 mf_direita($) = 1;
137
138 saida_regra_1 = min(forca_regra_1 * ones(theta_discreto), mf_esquerda);
139 saida_regra_2 = min(forca_regra_2 * ones(theta_discreto), mf_direita);
140 saida_manual = max([saida_regra_1; saida_regra_2], "r");
141
142 theta_manual = sum(theta_discreto .* saida_manual) / sum(saida_manual);
143
144 mprintf("Calculo manual (passo de 5 graus) = %.6f graus \n", theta_manual);
145 mprintf("Diferenca software - manual = %.6f graus \n \n", theta - theta_manual);
146
147 // =====
148 // 8. SALVAR O SISTEMA E O RESULTADO NUMERICO
149 // =====
150
151 savefls(fl, pasta_projeto + "fuzzy_robotica.fl");
152
153 fd = mopen(pasta_projeto + "resultado_simulacao.txt", "wt");
154 mfprintf(fd, "ATIVIDADE 3A - CONTROLE FUZZY PARA ROBOTICA MOVEL \n \n");
155 mfprintf(fd, "Entrada d = %.2f cm \n", d);
156 mfprintf(fd, "Entrada a = %.2f cm \n", a);
157 mfprintf(fd, "Angulo pelo sciFLT = %.6f graus \n", theta);
158 mfprintf(fd, "Angulo manual (passo de 5 graus) = %.6f graus \n", theta_manual);
159 mfprintf(fd, "Diferenca = %.6f graus \n", theta - theta_manual);
160 mfprintf(fd, "Acao = %s \n", acao);
161 mclose(fd);
162
163 // =====
164 // 9. GRAFICOS DAS FUNCOES DE PERTINENCIA
165 // =====
166
167 f1 = scf(1);
168 clf();
169 plotvar(fl, "input", 1, -0.1, 1.1, 1001);
170 xtitle("Distancia Frontal", "Distancia (cm)", "Grau de pertinencia");
171 xs2png(f1, pasta_prints + "01_distancia_frontal.png");
172
173 f2 = scf(2);

```

```

174 clf();
175 plotvar(fls, "input", 2, -0.1, 1.1, 1001);
176 xtitle("Assimetria Lateral", "Assimetria (cm)", "Grau de pertinencia");
177 xs2png(f2, pasta_prints + "02_assimetria_lateral.png");
178
179 f3 = scf(3);
180 clf();
181 plotvar(fls, "output", 1, -0.1, 1.1, 1001);
182 xtitle("Angulo de Direcao", "Angulo (graus)", "Grau de pertinencia");
183 xs2png(f3, pasta_prints + "03_angulo_direcao.png");
184
185 // Grafico da saida agregada e do valor defuzzificado.
186 universo_saida = linspace(-45, 45, n_pontos)';
187
188 f4 = scf(4);
189 clf();
190 plot(universo_saida, saida_agregada(:, 1), "b-", "LineWidth", 3);
191 plot([theta theta], [0 1], "r--", "LineWidth", 2);
192 xgrid();
193 xtitle(sprintf("Resultado: theta = %.4f graus - %s", theta, acao), ..
194         "Angulo (graus)", "Grau de pertinencia agregado");
195 xs2png(f4, pasta_prints + "resultado_simulacao.png");
196
197 // Superficie tridimensional do controlador fuzzy.
198 // Eixo X: distancia; eixo Y: assimetria; eixo Z: angulo calculado.
199 n_distancia = 31;
200 n_assimetria = 31;
201 // Evita somente os pontos extremos exatos, pois o trapmf antigo do sciFLT
202 // devolve zero quando os dois vertices do ombro coincidem no limite.
203 distancias = linspace(0.001, 99.999, n_distancia)';
204 assimetrias = linspace(-49.999, 49.999, n_assimetria)';
205 malha_entradas = genspace(distancias, assimetrias);
206 angulos_malha = zeros(size(malha_entradas, 1), 1);
207
208 // A versao 0.5 do sciFLT pode falhar com uma matriz grande em evalfls.
209 // A avaliacao ponto a ponto produz a mesma superficie de forma compativel.
210 for k = 1:size(malha_entradas, 1)
211     angulo_ponto = evalfls(malha_entradas(k, :), fls, 201);
212     angulos_malha(k) = angulo_ponto(1);
213 end
214 superficie_angulo = matrix(angulos_malha, n_assimetria, n_distancia)';
215
216 f5 = scf(5);
217 clf();
218 f5.figure_size = [1000 700];
219 f5.color_map = jetcolormap(64);
220 plot3d1(distancias, assimetrias, superficie_angulo);
221 xtitle("Superficie 3D do Controle Fuzzy", ..
222        "Distancia frontal (cm)", ..
223        "Assimetria lateral (cm)", ..
224        "Theta (graus)");
225 xs2png(f5, pasta_prints + "04_superficie_controle_3d.png");
226
227 // Graficos preenchidos da Parte 1 - Fase 1.
228 // Mostram os valores crisp e os graus de pertinencia calculados manualmente.
229 d_grade = linspace(0, 100, 1001)';
230 perto_grade = trapmf(d_grade, [0 0 15 40]);
231 media_grade = trimf(d_grade, [15 50 85]);
232 longe_grade = trapmf(d_grade, [60 85 100 100]);
233 perto_grade(1) = 1;
234 longe_grade($) = 1;
235
236 a_grade = linspace(-50, 50, 1001)';
237 negativa_grade = trapmf(a_grade, [-50 -50 -25 0]);
238 zero_grade = trimf(a_grade, [-25 0 25]);
239 positiva_grade = trapmf(a_grade, [0 25 50 50]);
240 negativa_grade(1) = 1;
241 positiva_grade($) = 1;
242
243 f6 = scf(6);
244 clf();
245 f6.figure_size = [1200 520];
246 subplot(1, 2, 1);
247 plot(d_grade, perto_grade, "b-", "LineWidth", 2);
248 plot(d_grade, media_grade, "g-", "LineWidth", 2);
249 plot(d_grade, longe_grade, "r-", "LineWidth", 2);
250 plot([d d], [0 1.05], "k--", "LineWidth", 2);
251 plot(d, mu_perto, "ko", "MarkerSize", 7);
252 xgrid();
253 gca().data_bounds = [0 -0.05; 100 1.08];
254 xtitle("Fase 1: d = 10 cm", "Distancia frontal (cm)", "Grau de pertinencia");
255 xstring(17, 0.92, "muPerto = 1,0");
256 xstring(17, 0.82, "muMedia = 0,0; muLonge = 0,0");
257
258 subplot(1, 2, 2);
259 plot(a_grade, negativa_grade, "b-", "LineWidth", 2);
260 plot(a_grade, zero_grade, "g-", "LineWidth", 2);
261 plot(a_grade, positiva_grade, "r-", "LineWidth", 2);

```

```

262 plot([a a], [0 1.05], "k--", "LineWidth", 2);
263 plot(a, mu_negativa, "bo", "MarkerSize", 7);
264 plot(a, mu_zero, "go", "MarkerSize", 7);
265 xgrid();
266 gca().data_bounds = [-50 -0.05; 50 1.08];
267 xtitle("Fase 1: a = -10 cm", "Assimetria lateral (cm)", "Grau de pertinencia");
268 xstring(-47, 0.92, "muNegativa = 0,4");
269 xstring(2, 0.72, "muZero = 0,6");
270 xstring(2, 0.62, "muPositiva = 0,0");
271 xs2png(f6, pasta_prints + "05_fase1_entradas_annotadas.png");
272
273 // Fases 3 e 4: implicacao de Mamdani e agregacao pelo maximo.
274 theta_fino = linspace(-45, 45, 901)';
275 esquerda_fina = trapmf(theta_fino, [-45 -45 -20 0]);
276 direita_fina = trapmf(theta_fino, [0 20 45 45]);
277 esquerda_fina(1) = 1;
278 direita_fina($) = 1;
279 regra_1_fina = min(forca_regra_1 * ones(theta_fino), esquerda_fina);
280 regra_2_fina = min(forca_regra_2 * ones(theta_fino), direita_fina);
281 agregada_fina = max([regra_1_fina regra_2_fina], "c");
282
283 f7 = scf(7);
284 clf();
285 f7.figure_size = [1350 520];
286
287 subplot(1, 3, 1);
288 plot(theta_fino, regra_1_fina, "b-", "LineWidth", 3);
289 xgrid();
290 gca().data_bounds = [-45 -0.05; 45 0.82];
291 xtitle("Fase 3 - Regra 1", "Theta (graus)", "Pertinencia");
292 xstring(-40, 0.47, "Virar Esquerda: alfa1 = 0,4");
293
294 subplot(1, 3, 2);
295 plot(theta_fino, regra_2_fina, "r-", "LineWidth", 3);
296 xgrid();
297 gca().data_bounds = [-45 -0.05; 45 0.82];
298 xtitle("Fase 3 - Regra 2", "Theta (graus)", "Pertinencia");
299 xstring(2, 0.67, "Virar Direita: alfa2 = 0,6");
300
301 subplot(1, 3, 3);
302 plot(theta_fino, agregada_fina, "k-", "LineWidth", 3);
303 plot([theta_manual theta_manual], [0 0.72], "m--", "LineWidth", 2);
304 xgrid();
305 gca().data_bounds = [-45 -0.05; 45 0.82];
306 xtitle("Fase 4 - Agregacao (maximo)", "Theta (graus)", "Pertinencia");
307 xstring(-42, 0.75, msprintf("CDA = %.4f graus", theta_manual));
308 xs2png(f7, pasta_prints + "06_fases3_4_implicacao_agregacao.png");
309
310 mprintf("Arquivos salvos em: %s\n", pasta_projeto);
311 mprintf("Sistema salvo como fuzzy_robotica.fls\n");
312 mprintf("Graficos das 5 fases, simulacao e superficie 3D salvos em prints.\n");

```